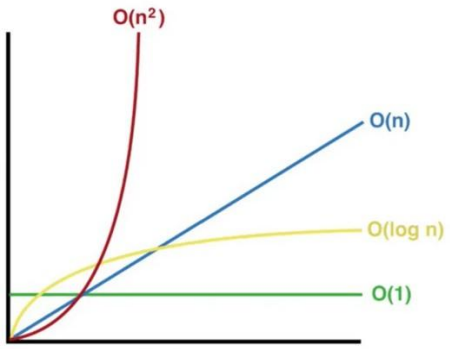
Disciplina: Estrutura de Dados



Análise de Complexidade

A análise de complexidade de algoritmos é essencial para o desenvolvimento de software eficiente e escalável. Ao entender como um algoritmo se comporta em termos de tempo de execução e uso de memória, os desenvolvedores podem tomar decisões informadas para otimizar o desempenho de seus programas.

Isso é especialmente crítico em aplicações que processam grandes volumes de dados ou que exigem respostas rápidas, como sistemas financeiros, inteligência artificial e jogos.

1) Considere o seguinte código. Qual é a complexidade de tempo desse algoritmo em notação Big-O?

```
public class Questao1 {
    public static void main(String[] args) {
        int n = 10;
        for (int i = 0; i < n; i++) {
            System.out.println("Iteração: " + i);
        }
    }
}
```

O código possui um único laço de repetição for que incrementa de 1 em 1 iterando de 0 a n-1. A complexidade de tempo é O(n).

2) Qual é a complexidade de tempo do seguinte algoritmo:

```
public class Questao2 {
    public static void main(String[] args) {
        int n = 10;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.println("Iteração: " + i + ", " + j);
```

```
            }
        }
    }
}
```

Existem dois laços de repetição for aninhados independentes, ambos iterando de 0 até n-1. A complexidade de tempo é O(n^2).

3) Analise a complexidade de tempo do algoritmo a seguir:

```
public class Questao3 {
    public static void main(String[] args) {
        int n = 10;
        int i = 1;
        while (i < n) {
            System.out.println("i: " + i);
            i *= 2;
        }
    }
}
```

O laço while multiplica a variável i por 2 a cada iteração (i *= 2) até que o valor atinja n. A complexidade de tempo é O(log n).

4) Determine a complexidade de tempo do seguinte algoritmo:

```
public class Questao4 {
    public static void main(String[] args) {
        int n = 10;
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                System.out.println("Iteração: " + i + ", " + j);
            }
        }
    }
}
```

O código apresenta dois laços aninhados onde o laço externo vai de 0 a n-1 e o interno vai de i a n-1. A soma das iterações é equivalente a uma progressão aritmética, resultando na complexidade O(n^2).

5) Qual é a complexidade de tempo do seguinte código?

```
public class Questao5 {
    public static void main(String[] args) {
        int n = 10;
        for (int i = 0; i < n; i += 2) {
            System.out.println("Iteração: " + i);
        }
    }
```

```
}
```
O laço for itera de 0 até n-1, mas incrementa o contador de 2 em 2 (i += 2). Como o laço executa n/2 vezes, a complexidade é O(n).

6) Analise a complexidade do seguinte algoritmo:

```
public class Questao6 {
    public static void main(String[] args) {
        int n = 10;
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += i;
        }
        System.out.println("Soma: " + sum);
    }
}
```
Há apenas um laço for percorrendo os valores de 0 a n-1 para realizar uma operação constante de soma. A complexidade é O(n).

7) Qual é a complexidade do algoritmo a seguir:

```
public class Questao7 {
    public static void main(String[] args) {
        int n = 10;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < Math.√(n); j++) {
                System.out.println("Iteração: " + i + ", " + j);
            }
        }
    }
}
```
O algoritmo possui dois laços: o externo roda n vezes e o interno roda até a raiz quadrada de n (√n). A complexidade resultante é O(n √n).

8) Determine a complexidade do seguinte código:

```
public class Questao8 {
    public static void main(String[] args) {
        int n = 10;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                for (int k = 0; k < n; k++) {
                    System.out.println("Iteração: " + i + ", " + j
+ ", " + k);
                }
            }
```

```
        }
    }
}
```

O código conta com três laços for aninhados, e todos iteram exatamente n vezes. A complexidade é O(n^3).

9) Qual é a complexidade do algoritmo:

```
public class Questao9 {
    public static void main(String[] args) {
        int n = 10;
        System.out.println(buscaLinear(new int[]{1, 2, 3, 4, 5, 6,
7, 8, 9, 10}, 5));
    }

    public static boolean buscaLinear(int[] arr, int x) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == x) {
                return true;
            }
        }
        return false;
    }
}
```

A função buscaLinear possui um único laço que percorre os elementos do array em busca de um valor correspondente. No pior caso, o laço irá visitar todos os n elementos, portanto a complexidade é O(n).

10) Analise a complexidade de tempo do código:

```
public class Questao10 {
    public static void main(String[] args) {
        int n = 10;
        fibonacci(n);
    }

    public static int fibonacci(int n) {
        if (n <= 1) {
            return n;
        }
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

A função recursiva fibonacci realiza duas novas chamadas para si mesma a cada execução que não cai no caso base (n <= 1). Isso gera uma árvore de recursão, resultando em uma complexidade de tempo exponencial de O(2^n).

11) Determine a complexidade de tempo do seguinte algoritmo:

```
public class Questao11 {
    public static void main(String[] args) {
        int n = 10;
        for (int i = 0; i < n; i++) {
            for (int j = i; j > 0; j /= 2) {
                System.out.println("Iteração: " + i + ", " + j);
            }
        }
    }
}
```

O laço externo é linear executando n vezes, e o laço interno, que se inicia em i, divide seu próprio contador por 2 a cada iteração (j /= 2). A soma dessas operações logarítmicas sucessivas resulta em uma complexidade de O(n log n).

12) Qual é a complexidade de tempo do código:

```
public class Questao12 {
    public static void main(String[] args) {
        int n = 10;
        int count = 0;
        for (int i = 1; i < n; i *= 3) {
            count++;
        }
        System.out.println("Count: " + count);
    }
}
```

O laço for é iniciado em 1 e o iterador é multiplicado por 3 a cada passo (i *= 3) até atingir n. Como o crescimento é geométrico, a complexidade é O(log n).

13) Analise a complexidade do seguinte código:

```java
public class Questao13 {
    public static void main(String[] args) {
        int n = 10;
        int sum = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < i; j++) {
                sum++;
            }
        }
        System.out.println("Soma: " + sum);
    }
}
```

O código possui um laço externo de 0 a n-1 e um laço interno que vai de 0 a i-1. O comportamento da somatória de iterações é similar ao da Questão 4, definindo a complexidade de tempo em O(n^2).

14) Determine a complexidade de tempo do algoritmo recursivo:

```java
public class Questao14 {
    public static void main(String[] args) {

        int n = 10;
        System.out.println(fatorial(n));
    }

    public static int fatorial(int n) {
        if (n == 0) return 1;
        return n * fatorial(n - 1);
    }
}
```

A função recursiva fatorial apenas decrementa o valor de n em 1 a cada nova chamada recursiva. A profundidade máxima da pilha de chamadas será n, fazendo com que a complexidade seja O(n).

15) Determine a complexidade de tempo do seguinte algoritmo:

```java
public class Questao15 {
    public static void main(String[] args) {
        int n = 10;
        System.out.println(exp(2, n));
    }

    public static int exp(int base, int exp) {
        if (exp == 0) return 1;
        if (exp % 2 == 0) return exp(base * base, exp / 2);
        return base * exp(base, exp - 1);
    }
}
```

O método recursivo de exponenciação divide o valor de exp pela metade quando ele é par e o subtrai em 1 quando ímpar. Esse comportamento de redução drástica exige um número logarítmico de passos para que exp chegue a 0, logo a complexidade em relação ao expoente é O(log n).